



Memory Systems for Agents

AICB-P2T3 · Ngày 17 · Chương 4 — Agent Nâng Cao

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 4

HÃY SUY NGHĨ...

*“Tại sao agent của bạn quên
mọi thứ sau mỗi conversation –
và làm sao fix nó đúng cách?”*

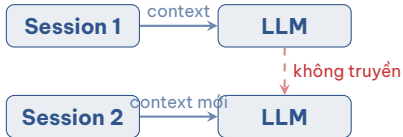
Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Tại sao Agent “quên”?
2. Context Engineering Framework
3. Cognitive Memory Model – 4 loại Memory
4. Implementation Deep-Dive
5. Frameworks chuyên dụng & Privacy
6. Xu hướng 2025–2026 trong Agent Memory
7. Demo & Thực hành

Tại sao Agent “quên”?

Context window có giới hạn – và hầu hết agent không có bộ nhớ ngoài

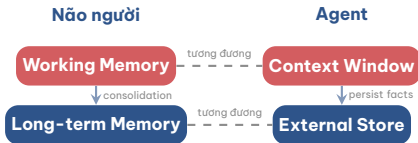
01



Mỗi session bắt đầu từ zero

- **LLM không có persistent state**
– mỗi API call là một request độc lập
- User nói “tôi thích Python” ở session 1 → session 2 agent **không nhớ**
- Conversation dài >50 turns → **hit context limit**

Lưu ý: Đây là vấn đề #1 khi deploy agent thực tế: user kỳ vọng agent “nhớ” – nhưng nó không.



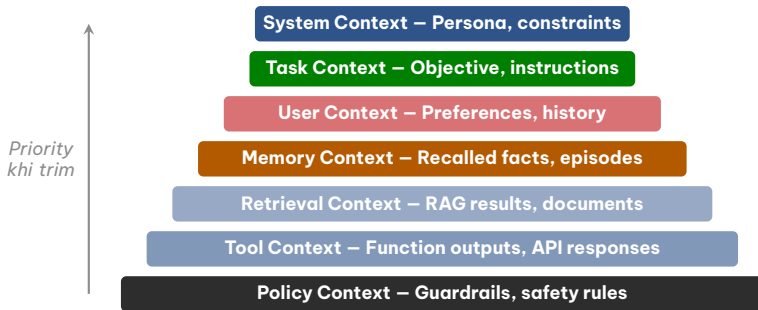
Context Window = RAM — Nhanh, tạm thời, giới hạn dung lượng (~128K tokens)

External Store = Ổ cứng — Chậm hơn, bền vững, gần như vô hạn (Redis, Vector DB)

Agent cần cả hai: **fast access** cho conversation hiện tại + **persistent storage** cho knowledge qua sessions

Context Engineering Framework

7 layers of context – quản lý những gì agent “thấy”



Khi gần token limit: **trim từ dưới lên**. Policy context trim cuối cùng (safety không bao giờ bỏ).

Lưu ý: Conflict resolution: user preference mâu thuẫn policy constraint → policy **luôn thắng**. Cần explicit rules trong system design.

10%

Short-term
memory

4%

Long-term
facts

3%

Episodic
memory

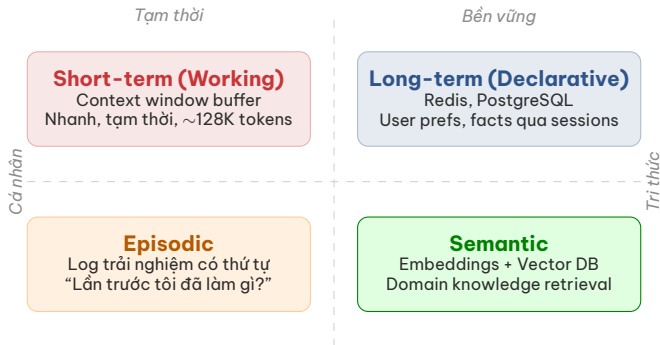
3%

Semantic
knowledge

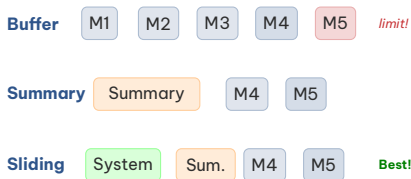
Phần còn lại dành cho system prompt, task instructions, tool outputs, và output generation. Vượt 20% → context bị nhiễu, accuracy giảm.

Cognitive Memory Model – 4 loại Memory

Short-term, Long-term, Episodic, Semantic



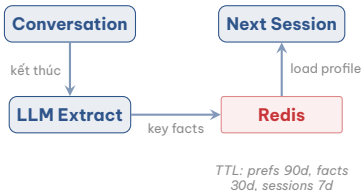
Working memory ↔ Short-term | Declarative memory ↔ Long-term
| Episodic & Semantic tương tự tên gọi



3 strategies chính:

1. **Buffer:** giữ tất cả – đơn giản nhưng hit limit sau ~ 50 turns
2. **Summary:** LLM tóm tắt history cũ – ổn định nhưng tốn thêm LLM calls
3. **Sliding window:** system + summary + last K turns – *best tradeoff cho production*

Short-term memory nên chiếm tối đa **10% context window**. Trim khi vượt – keep recent N tokens, discard oldest.



Ý tưởng:

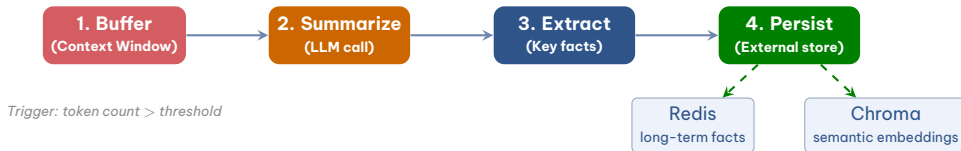
- Sau mỗi conversation, LLM **extract key facts** rồi store vào Redis với TTL
- Session mới: load user profile vào system prompt *trước* khi user nói

Hash: preferences (language, style)

Set: facts (“biết Python, học ML”)

List: session history (recent)

Tất cả **O(1) reads** – production-ready



Chỉ persist sau **task completion** – không write giữa chừng để tránh inconsistent state

Lưu ý: Conflict resolution: long-term fact mâu thuẫn short-term info → **recency wins**, flag for re-view.

Implementation Deep-Dive

Code-level: LangGraph nodes cho mỗi memory type

```
class MemoryState(TypedDict):
    messages: list[BaseMessage]
    user_profile: dict          # long-term
    episodes: list[dict]        # episodic
    semantic_hits: list[str]    # semantic
    memory_budget: int          # tokens left

# Memory router: chọn loại phù hợp
def retrieve_memory(state):
    query = state["messages"][-1].content
    return {
        "user_profile": redis.hgetall(uid),
        "episodes": find_similar(query, k=3),
        "semantic_hits": chroma.query(query),
    }
```

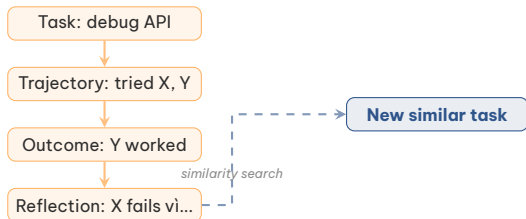
Integration pattern:

1. Node `load_memory`: đọc 3 loại memory khi bắt đầu
2. Inject vào system prompt theo priority
3. Node `save_memory`: ghi khi kết thúc

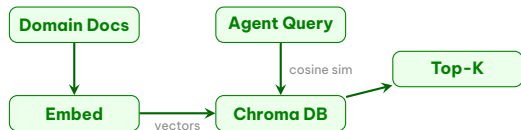
1. Short-term (gần nhất)
2. Long-term facts (user prefs)
3. Relevant episodes
4. Semantic knowledge

Lưu tuple mỗi episode:

- (task, trajectory, outcome, reflection)
- Agent biết: “approach X đã fail vì Y trong task tương tự”

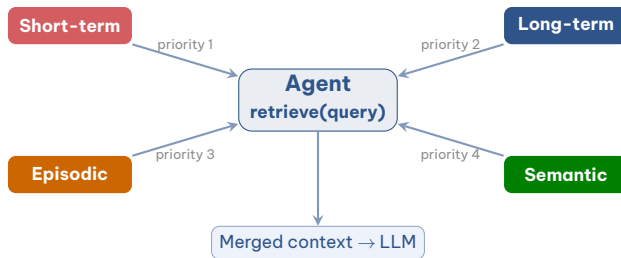


LRU: xóa episode ít dùng nhất
Importance decay: score giảm theo thời gian
Consolidation: merge episodes tương tự
Voyager-style: extract reusable strategy → skill library



- Encode domain knowledge → embeddings → Chroma/Pinecone
- Query = task description → cosine similarity → top-k chunks
- Agent discover facts mới → add vào DB với metadata (source, confidence, timestamp)

Agent tự mở rộng knowledge base qua interactions – incremental knowledge growth



Lưu ý: Unified interface: `retrieve(query, types=["all"])` trả về merged context từ cả 4 loại memory, đã trim theo token budget.

Frameworks chuyên dụng & Privacy

Mem0, Zep – và khi nào dùng framework có sẵn

- Auto-classify memory types
- Smart retrieval: relevance + recency ranking
- Claim: **90% token reduction**, 91% faster retrieval
- API-first, nhanh go-to-market

- Entity extraction + progressive summarization
- Tự build user knowledge graph qua sessions
- Multi-level summaries: turn → session → cross-session
- Giảm context size tối ưu

Tiêu chí	MemO / Zep	Custom (Redis + Chroma)
Setup time	Nhanh (API)	Chậm (build from scratch)
Control	Hạn chế	Full control
Khi nào dùng	MVP, go-to-market	Production, đặc thù domain

Privacy-by-Design – Mặc định **không lưu PII**. User phải explicit opt-in trước khi agent ghi nhớ thông tin cá nhân.

Right to be Forgotten – User yêu cầu xóa → xóa **tất cả** memory entries liên quan → confirm deletion.

Lưu ý: Federated Forgetting: trong multi-agent system, deletion request phải **propagate** đến tất cả agents có copy.

- ✓ Data minimization
- ✓ Purpose limitation
- ✓ Storage limitation (TTL)
- ✓ Consent management
- ✓ Deletion verification

Xu hướng 2025–2026 trong Agent Memory

Từ persistent profiles đến file-based identity, heartbeat loops và compiled knowledge bases

1. Cross-session memory – Từ thread history sang user-scoped profile, open loops, episodic store.

2. Compaction + notes – Tóm tắt phần đã xử lý, giữ recent context, đẩy facts / tasks ra durable notes.

3. Identity files – AGENTS.md, CLAUDE.md, SOUL.md trở thành lớp control plane.

4. Heartbeat loops – Agent được đánh thức định kỳ để refresh state, xử lý back-log, làm sạch notes.

5. Compiled knowledge base – Kiểu Karpathy *LLM Wiki*: tích lũy wiki đã tổng hợp thay vì RAG lại từ documents thô.

Vì sao pattern này bùng lên? –

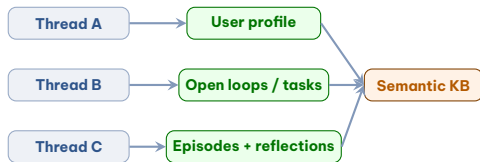
User không còn chấp nhận agent “mất trí nhớ” sau mỗi thread. Với product assistant, sales copilot, tutor, coding agent, experience phải liên tục qua nhiều ngày.

Memory hiện đại thường lưu **preference, decision, open loop, episodic outcome** – chứ không lưu toàn bộ transcript.

Những thay đổi quan trọng:

- Từ **thread-scoped state** sang **user-scoped memory namespace**
- Từ “save chat history” sang **save structured facts**
- Từ retrieval chỉ dựa trên similarity sang **scope + recency + relevance + provenance**
- Từ write-back cuối phiên sang **hot path + background consolidation**

Sai lầm phổ biến: đem toàn bộ transcript qua session mới. Kết quả là token phình to, recall nhiều và rủi ro rò rỉ dữ liệu giữa user tăng mạnh.



*Namespacing theo user_id, org_id,
topic – không dump full transcript*

- **Short-term** vẫn thread-scoped; **long-term** thường user-scoped hoặc org-scoped
- Persist **facts, decisions, preferences, open loops**; đừng lưu raw chat vô tội vạ
- Memory record nên có source, timestamp, confidence, ttl
- Deletion và consent phải chạy xuyên suốt mọi store, không chỉ store chính

Memory gán nhầm user là bug nghiêm trọng. Isolation, TTL, deletion và provenance phải có từ đầu.

Vấn đề gốc – Conversation dài làm context window bị lấp bởi phần đã “xử xong”. Agent vẫn đọc lại lịch sử, nhưng phần lớn token không còn tạo giá trị cho lượt trả lời hiện tại.

Lưu ý: Nếu không compact

Latency tăng, chi phí tăng, retrieval kém chính xác hơn, và output dễ lẫn giữa việc “đã quyết định” với việc “đang thảo luận”.

Compaction thường giữ lại 3 lớp:

1. **Recent turns:** phần hội thoại mới nhất còn đang nóng
2. **Session summary:** tóm tắt trạng thái hiện tại, quyết định, blockers
3. **Durable notes:** facts / tasks đủ quan trọng để chuyển sang long-term memory

Agent chỉ mang theo những gì còn **phục vụ lượt suy luận tiếp theo**, thay vì kéo cả transcript đi cùng.



Không compact mọi lúc. Chỉ chạy khi gần token limit, khi session kéo dài, hoặc khi chuẩn bị chuyển pha từ thảo luận sang thực thi.

Lưu ý: Bẫy thường gặp

Summary mơ hồ hơn transcript gốc. Vì vậy notes phải ưu tiên **trạng thái, quyết định, TODO, constraint** thay vì tóm tắt lan man.

File	Vai trò phù hợp
AGENTS.md	Quy tắc làm việc, workflow, boundaries, coding / execution norms
SOUL.md	Identity, tone, preferred defaults, behavioral profile
MEMORY.md	Notes bền vững, memory schema, recap các quyết định quan trọng
TASKS.md	Open loops, backlog, agenda đang theo dõi

Vai trò đúng – Đây là **control plane**: giúp phiên mới bootstrap nhanh, giữ hành vi nhất quán và giảm việc phải “nhắc lại prompt gốc”.

Lưu ý: Đừng hiểu sai

Những file này **không phải retrieval engine**. Chúng không tự re-rank, không TTL, không conflict resolution, không provenance.

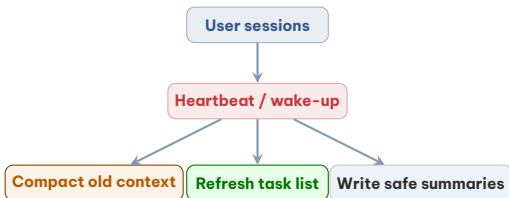
Dùng AGENTS.md / SOUL.md để cố định persona, workflow, default actions và memory schema.

Dùng structured memory store hoặc vector / graph / KV store để recall preference, open loops, episode, knowledge khi thật sự liên quan.

Thiết kế nên có:

- Một file **instruction** đủ ngắn để nạp đầu phiên
- Một file **working notes** để agent cập nhật có kiểm soát
- Một **memory backend** riêng cho recall theo scope / relevance
- Chính sách rõ về **ai được ghi gì, khi nào ghi, ghi vào đâu**

Instruction files trả lời: “hãy xử lý thế nào”; retrieval memory trả lời: “nên nhớ điều gì lúc này”.



- Hợp với personal assistant, inbox triage, research copilot, coding agent có backlog dài ngày
- Background pass giảm áp lực phải “ghi nhớ ngay lập tức” ở cuối mỗi chat
- Có thể dùng để de-duplicate notes, re-rank backlog, expire stale tasks, tạo daily recap

Agent chủ động duy trì state ngay cả khi user không mở chat mới. Đây là bước chuyển từ chatbot phản ứng sang agent có vòng đời.

Lưu ý: Rủi ro chính

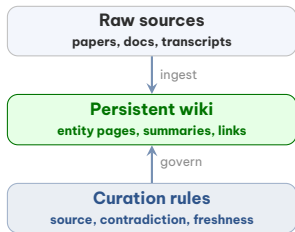
Heartbeat + persistent memory khiến prompt injection không chỉ phá một lượt chat mà còn có thể trở thành **memory poisoning** hoặc backlog poisoning kéo dài qua nhiều phiên.

Các write nguy hiểm – Ghi lại instruction mới, thay đổi preference nhạy cảm, tạo task tự chạy, hoặc lưu “fact” không có provenance.

Guardrails tối thiểu:

- Allowlist rõ **memory types** nào được phép ghi
- Gắn **source, timestamp, confidence** cho mọi durable write
- Tách **user request, system rule, agent inference** thành các trường riêng
- High-impact writes phải qua **human review** hoặc policy check

Heartbeat không được tự cho mình thêm quyền. Nó chỉ là cơ chế vận hành nền, không phải cánh cửa để bypass policy.



- Core idea: LLM **tích lũy knowledge thành wiki liên kết**, thay vì rediscover từ raw docs cho mỗi query
- Mỗi page đại diện cho một entity, concept, timeline, contradiction hoặc decision summary
- Hợp với research, due diligence, course notes, internal KB – nơi provenance và synthesis quan trọng

Knowledge ngày càng “được biên tập” tốt hơn theo thời gian, thay vì mỗi lượt hỏi lại bắt đầu từ documents thô.

Plain RAG đủ tốt khi nào? – Hỏi đáp 1 lần trên corpus lớn, tài liệu đổi liên tục, hoặc team chưa cần curate tri thức theo entity / timeline / contradiction.

Khi cùng một corpus được hỏi lặp đi lặp lại trong nhiều tuần và team cần tri thức đã được chứng cất dần, có link, có provenance, có recap.

Tín hiệu nên nâng từ RAG lên KB:

- Câu hỏi lặp lại nhưng vẫn phải retrieval / synthesis lại từ đầu
- Team cần theo dõi **timeline**, **contradiction**, **entity pages**, **decision history**
- Người dùng cần **audit trail** thay vì chỉ top-k chunks

*Nếu team hỏi lặp lại những câu hỏi phức tạp trên cùng một tập tài liệu trong nhiều tuần, hãy cân nhắc **compiled KB** thay vì chỉ tăng top-k cho RAG.*

Demo & Thực hành

Xem agent nhớ user preferences qua 3 sessions

07

1. **Session 1:** User nói “tôi thích Python, không thích Java” → agent ghi vào Redis
2. **Session 2** (new process): Agent load memory → proactively suggest Python solution mà không cần hỏi lại
3. **Session 3:** Agent recall episode “user bị confused async/await” → tự thêm explanation
4. So sánh: agent có memory vs không memory – response relevance, user satisfaction

Mục tiêu: Build Multi-Memory Agent với LangGraph

Deliverable: Agent với full memory stack + benchmark report: so sánh agent có/không memory trên 10 multi-turn conversations

Thời gian: 2 giờ

1. **Implement 4 memory backends:** ConversationBufferMemory (short-term), Redis (long-term), JSON episodic log, Chroma (semantic)
2. **Build memory router:** chọn memory type phù hợp dựa trên query intent – user preference vs factual recall vs experience recall
3. **Context window management:** auto-trim khi gần limit, priority-based eviction theo 4-level hierarchy
4. **Benchmark:** so sánh agent có/không memory trên 10 multi-turn conversations – đo response relevance, context utilization, token efficiency

GitHub repo + benchmark report: bảng so sánh metrics, memory hit rate analysis, token budget breakdown

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Không có “one size fits all” – production agent cần ít nhất **short-term + long-term**, thêm episodic/semantic tùy use case
- 2 Memory retrieval quality quyết định agent quality – bad retrieval = irrelevant context = wrong answer
- 3 Memory write-back cần careful design: nhớ gì, khi nào ghi, xử lý conflict ra sao, TTL bao lâu
- 4 Privacy không phải afterthought – GDPR compliance cần thiết kể từ đầu (Privacy-by-Design)

Ngày 18: Production RAG

“Agent đã có memory, tiếp theo cần knowledge retrieval tốt hơn – tại sao RAG pipeline demo chạy tốt nhưng production chỉ đạt 60%?”

- Hoàn thành Lab 17: Multi-Memory Agent + benchmark
- Đọc: Anthropic “Building Effective Agents” (mục Context Engineering)

Hỏi & Đáp

*Memory nào là “must-have” cho production agent?
Khi nào thì dùng framework (Mem0, Zep) vs tự build?*



Cảm ơn!

AICB-P2T3 · Ngày 17 · Memory Systems for Agents

`github.com/vinuni-aicb`

Liên hệ: `instructor@vinuni.edu.vn`